CS 320 – Software Test, Automation QA

Alexis Lewis

Professor Karl Lewis

October 18th, 2026

Project 7-2

Summary of Unit Testing Approach and Requirements

For Project One, I utilized JUnit 5 to test the contact, task, and appointment objects, as well as their respective service classes, using separate test classes. My approach was requirements-based: I verified that valid information could be stored and retrieved, invalid information was rejected, and the services correctly handled updates and deletions. Across six test classes, I implemented 63 test methods: 20 for contacts, 19 for tasks, and 24 for appointments. This ensured coverage of both standard operations and various failure scenarios.

The contact tests addressed required field lengths, null values, and the requirement that a phone number must contain exactly ten digits. For instance, ContactTest.java, lines 132–150, evaluates phone numbers that are too short or contain non-digit characters. ContactServiceTest.java, lines 75–99, confirms that duplicate contact IDs are rejected. The service tests also verify updates to the first name, last name, phone, and address fields. These cases correspond directly to the contact service requirements, though they do not demonstrate exhaustive input coverage.

For tasks, TaskTest.java, lines 23–64, rejects null or oversized IDs, names, and descriptions. TaskServiceTest.java, lines 32–42, evaluates duplicate IDs, while lines 63–81 confirm that updating a name or description modifies the stored task. For appointments, AppointmentTest.java, lines 55–128, assesses invalid IDs, missing dates, past dates, and description length constraints. AppointmentServiceTest.java, lines 72–101, examines duplicate IDs and null appointments, and lines 130–150 confirm that deleting one appointment does not affect others. These tests directly link expected application behavior to specific assertions.

Evaluation of JUnit Test Quality, Technical Soundness, and Efficiency

I consider the test suite a solid foundation because it evaluates both valid inputs and anticipated failure cases. For example, TaskServiceTest.java, lines 39–41, employs assertThrows(IllegalArgumentException.class, ...) to confirm that duplicate tasks are rejected, rather than simply verifying code execution. AppointmentTest.java, lines 200–217, ensures that modifications to Date objects do not inadvertently alter the stored appointment date. These assertions address meaningful requirements and facilitate clearer interpretation of failures.

Although my JUnit tests passed successfully, my submitted project did not include the code coverage report. Therefore, I cannot provide an exact line or branch coverage percentage. However, the successful test results, combined with the 63 test methods covering valid inputs, invalid inputs, and service operations, provide evidence that the application meets its tested requirements. While these results support the effectiveness of my testing approach, additional coverage analysis would help identify any untested portions of the code.

I aimed to maintain test efficiency by employing focused methods and repeatable setup procedures. TaskServiceTest.java, lines 18–21, utilizes @BeforeEach to instantiate a new TaskService for each test, ensuring that prior test data does not influence subsequent results. AppointmentServiceTest.java, lines 18–26, applies a seeded Random object to generate consistent future-date test inputs, and lines 178–224 evaluate multiple appointments. These strategies minimize redundant setup and enhance reproducibility. To further improve the suite, I would incorporate precise boundary tests, including values just below and above each maximum, and implement fixed or injectable clock values for date-sensitive tests.

Reflection on Testing Techniques and Their Applications

The primary techniques I utilized included unit testing, black-box requirements-based testing, positive and negative testing, and boundary-value testing. Unit testing isolates individual classes or services, enabling early identification of issues before system integration. Black-box testing assesses observable behavior against specified requirements without reference to internal implementation. Positive tests validate expected operations, such as adding a contact, while negative tests verify that prohibited actions, such as inserting a duplicate ID, are correctly rejected. Boundary testing examines input limits, such as the ten-character appointment ID and fifty-character description.

These techniques are particularly effective for small back-end services with well-defined validation rules. For instance, rejecting an invalid phone number at the object level prevents erroneous data from entering the contact service. I also employed reproducible, seeded test data in the appointment suite to evaluate multiple add, retrieve, and delete operations. While this

approach broadens the range of tested scenarios, it does not substitute for carefully selected edge cases or performance assessments.

Other techniques not demonstrated in this project include integration testing, system testing, performance or load testing, security testing, and user acceptance testing. Integration tests evaluate interactions among components, such as between a service and a database, while system tests validate the complete application from end to end. Performance testing assesses response time and behavior under high load, and security testing examines resistance to unauthorized access and malicious inputs. User acceptance testing determines whether the application meets the customer's practical requirements. These methods gain importance when services are integrated with interfaces, networks, or production data stores.

Reflection on Caution, Bias, and Professional Discipline

I approached testing with caution by verifying both expected successes and anticipated failures. A service that accepts valid objects may still be unreliable if it permits duplicate identifiers or inadvertently deletes unrelated records. AppointmentServiceTest.java, lines 130–150, exemplifies this approach: the test confirms that only the targeted appointment is deleted, while others remain unaffected. I also considered the relationship between object and service classes. Constructor-level validation safeguards object creation, but service-level tests must independently verify unique IDs, retrieval, and deletion operations.

Testing my own code introduces the risk of confirmation bias, as I am aware of my intended implementation. To mitigate this bias, I developed tests for null inputs, invalid lengths,

duplicate IDs, and missing records, rather than focusing solely on successful operations. An independent reviewer or a comprehensive requirements checklist could help identify overlooked assumptions. For instance, further tests could examine empty strings, boundary lengths, null task inputs, and behavior near the current time. The current suite should not be considered exhaustive.

Professional discipline requires resisting the urge to remove failing tests or bypass validation to meet deadlines. Even minor shortcuts can result in technical debt if defects propagate into subsequent features and become more difficult to resolve. In future projects, I intend to maintain a requirements-to-tests checklist, execute the JUnit suite after each change, utilize a coverage tool to identify untested code, and review both passing and failing results. I will also refactor duplicated test setup and introduce regression tests whenever a bug is identified. Consistent testing is not merely about achieving passing results; it is about providing reliable evidence that the software fulfills its requirements.